Programming Assignment 2

Distance Vector Routing Protocols

---

1. Problem Statement

In this assignment you will implement a simplified version of the *Distance Vector Routing Protocol*. The protocol will be run on top of four servers/machines/laptops (behaving as routers) **using UDP**. Each server runs on a machine at a pre-defined port number. The servers should be able to output their forwarding tables along with the cost and should be robust to link changes. A server should send out routing packets only in the following two conditions: **a) periodic update** and **b) the user uses command asking for one**. This is a little different from the original algorithm which immediately sends out update routing information when routing table changes.

2. Getting Started

A Distance Vector Routing Algorithm.

3. Protocol Specification

The various components of the protocol are explained step by step. Please follow the specifications as described in the following parts.

3.1 Topology Establishment

In this programming assignment, you will use four servers/machines/laptops. **The four machines are required to form a network topology as shown in Fig. 1**. Each server is supplied with a topology file at startup that it uses to build its initial routing table. The topology file is local and contains the link cost to the neighbors. For all other servers in the network, the initial cost would be infinity. Each server can only read the topology file for itself. The entries of a topology file are listed below:

- ***<num-servers>***
- ***<num-neighbors>***
- ***<server-ID> <server-IP> <server-port>***
- ***<server-ID1> <server-ID2> <cost>***

num-servers: total number of servers in the network.
num-neighbors: the number of directly linked neighbors of the server.
server-ID, server-ID1, server-ID2**:** a unique identifier for a server, which is assigned by you.
cost**:** cost of a given link between a pair of servers. Assume that cost is an integer value.
Here is an example, consider the topology in Figure 1. We give a topology file for server 1 as shown in the table below.

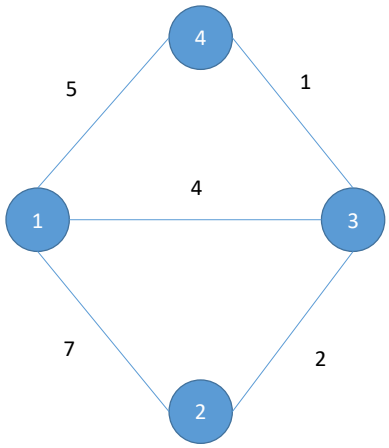


Figure 1: The network topology

Line number	Line entry	Comments
1	4	number of servers
2	3	number of edges or neighbors
3	1 128.205.36.8 4091	server-id 1 and corresponding IP, port pair
4	2 128.205.35.24 4094	server-id 2 and corresponding IP, port pair
5	3 128.205.36.24 4096	server-id 3 and corresponding IP, port pair
6	4 128.205.36.4 7091	server-id 4 and corresponding IP, port pair
8	1 2 7	server-id and neighbor id and cost
9	1 3 4	server-id and neighbor id and cost
10	1 4 5	server-id and neighbor and cost

Your topology files should only contain the Line entry part (2nd column in the pink color). In each line, every two elements (e.g., server-id and corresponding IP, corresponding IP and port number) should be separated by a **space**. For cost values, **each topology file should only contain the cost values of the host server's neighbors** (The host server here is the one which will read this topology file). Note that the IPs of servers may change when you are running the program in a wireless network environment. So, we need to use ifconfig or ipconfig to obtain the IP first and then set up the topology file before the demo.

IMPORTANT: In this environment, costs are bi-directional i.e. the cost of a link from A-B is the same for B-A. Whenever a new server is added to the network, it will read its topology file to determine who its neighbors are. Routing updates are exchanged periodically between neighboring servers. When this newly added server sends routing messages to its neighbors, they will add an entry in their routing tables corresponding to it. Servers can also be removed from a network. When a server has been removed from a network, it will no longer send distance vector updates to its neighbors. When a server no longer receives distance vector updates from its neighbor for three consecutive update intervals, it assumes that the neighbor no longer exists in the network and makes the

appropriate changes to its routing table (link cost to this neighbor will now be set to infinity but not remove it from the table). This information is propagated to other servers in the network with the exchange of routing updates. Please note that although a server might be specified as a neighbor with a valid link cost in the topology file, the absence of three consecutive routing updates from this server will imply that it is no longer present in the network.

3.2 Routing Update

Routing updates are exchanged periodically between neighboring servers based on a time interval specified at the startup. In addition to exchanging distance vector updates, servers must also be able to respond to user-specified events. There are 4 possible events in this system. They can be grouped into three classes: topology changes, queries and exchange commands: (1) Topology changes refer to an updating of link status (update). (2) Queries include the ability to ask a server for its current routing table (display), and to ask a server for the number of distance vectors it has received (packets). In the case of the packets command, the value is reset to **zero** by a server after it satisfies the query. (3) Exchange commands can cause a server to send distance vectors to its neighbors immediately.

3.3 Message Format

Routing updates are sent using the General Message format. All routing updates use UDP messages. The message format for the data part is:

Number of update fields	Server port
Server IP	
Server IP address 1	
Server port 1	0x0
Server ID 1	Cost 1
Server IP address 2	
Server port 2	0x0
Server ID 2	Cost 2
….	

- **Number of update fields:** Indicate the number of entries that follow.
- **Server port:** port of the server sending this packet.
- **Server IP:** IP of the server sending this packet.
- **Server IP address:** IP of the n-th server in its routing table.
- **Server port:** port of the n-th server in its routing table.
- **Server ID:**server id of the n-th server on the network.
- **Cost:** cost of the **path** from the server sending the update to the n-th server whose ID is given in the packet.

Note: the servers listed in the packet can be any order i.e., 5, 3, 2, 1, 4.

4. Server Commands/Input Format

The server must support the following command at startup:

- **server -t <*topology-file-name*> -i <*routing-update-interval*>**

  topology-file-name: The topology file contains the initial topology configuration for the server, e.g., timberlake_init.txt. Please adhere to the format described in 3.1 for your topology files.

  routing-update-interval: It specifies the time interval between routing updates in seconds.

  port and server-id: They are written in the topology file. The server should find its port and server-id in the topology file without changing the entry format or adding any new entries.

The following commands can be specified at any point during the run of the server:

- **update <*server-ID1*> <*server-ID2*> <*Link Cost*>**

  server-ID1, server-ID2: The link for which the cost is being updated.

  Link Cost: It specifies the new link cost between the source and the destination server. Note that this command will be issued to **both** *server-ID1* and *server-ID2* and involve them to update the cost and no other server.

  For example:

  update 1 2 inf: The link between the servers with IDs 1 and 2 is assigned to infinity.

  update 1 2 8: Change the cost of the link to 8.

- **step**

  Send routing update to neighbors right away. Note that except this, routing updates only happen periodically.

- **packets**

  Display the number of distance vector packets this server has received since the last invocation of this information.

- **display**

  Display the current routing table. And the table should be displayed in a **sorted** order from small ID to big. The display should be formatted as a sequence of lines, with each line indicating: <destination-server-ID> <next-hop-server-ID> <cost-of-path>

- **disable <*server-ID*>**

  Disable the link to a given server. Doing this “closes” the connection to a given server with *server-ID*, e.g., set the corresponding link cost to infinity. Here you need to check if the given server is its neighbor.

- **crash**

  “Close” all connections. This is to simulate server crashes. Close all connections on all links. The neighboring servers must handle this close correctly and set the link cost to infinity.

5. Responses/Output Format

The following are a list of possible responses a user can see on their screen:

- On successful execution of an update, step, packets, display or disable command, the machine must display the following message:

  ***<command-string>* SUCCESS**

  where *command-string* is the command executed. Additional output as desired (e.g., for display, packets, etc. commands) is specified in the previous section.
- Upon encountering an error during execution of one of these commands, the machine must display the following response:

  <command-string> <error message>

  where *error message* is a brief description of the error encountered.
- On successfully receiving a route update message from neighbors, the machine must display the following response:

  RECEIVED A MESSAGE FROM SERVER *<server-ID>*

  where the *server-ID* is the id of the server which sent a route update message to the local server.

6. Submission and Grading

6.1 What to Submit

Your submission should contain a zip file – Name it as proj2.zip:

- All source files (.py files). Note: name your main program as dv.py.

6.2 How to submit

Use Canvas to submit the zip file.

6.3 Grading Criteria

- *Each group needs to show a demo during the office hour. All the members should attend the demo, and describe their coding contribution in this project.*
- Correctness of output. Please refer to the grading guidelines for more details.
- Organization and documentation of your code.

6.4 Grading Guidelines:

[+10] Server startup

- Server supports the given startup command format. [+5]
- Reads the topology file correctly. [+5]

[+15] Sending distance vector updates

- Sends the distance vector updates to neighbors only. [+5]

The server receiving an update message must print out the server ID from which it receives this message.

- Updates are sent in the specified format. [+5]

Please *clearly mark (using comments) the data structure of the update message you used for your implementation and mention the line number of where you define it in your report.*

- Updates are sent with an interval of specific time. [+5]

[+20] Routing table is updated correctly

- Maintains a routing table. [+5]

Please *clearly mark (using comments) the data structure of the routing table you used for your implementation and mention the line number of where you define it in your report.*

- Server reads the updates from neighbors correctly. [+5]
- Applies the Bellman Ford equation correctly. [+10]

[+45] Supports user commands

- update command [+10]
- step [+5]
- packets [+5]
- display [+5]
- disable <server-ID> [+5]
- crash [+10]
- Server responses to user commands are correct. [+5]

[+10] Code documentation and Report

- Code documentation and a short report explaining the overall implementation details [+10]